

CN WEEKLY LAB

LABORATORY REPORT

Linux File Permissions & Communication Tools

Ubuntu 24.04 LTS · Computer Networks

Student ID	23F3043
Lab No.	Weekly Lab 01
Platform	Ubuntu 24.04 LTS (VMware)
Subject	Computer Networks
Date	February 2026

LEARNING OBJECTIVES

1. Create and inspect files using the Linux terminal.
2. Understand and modify file permission bits (read, write, execute).
3. Use **chmod** (numeric & symbolic) to set access control.
4. Use **chown** to transfer file ownership via *sudo*.
5. Use **echo** and TTY redirection to send direct terminal messages.
6. Install and use the OpenSSH server to perform a local SSH login.

Task 1: Managing File Permissions

In this task we explore Linux's permission model. Every file carries three permission sets — **Owner**, **Group**, and **Others** — each with three bits: **read** (4), **write** (2), **execute** (1). We use **chmod** to change mode bits and **chown** to reassign ownership.

Step 1 — Create a Test File

We first create an empty file named `lab_report.txt` using the **touch** command. This is a safe practice file that will not interfere with any system files.

```
$ touch lab_report.txt
```

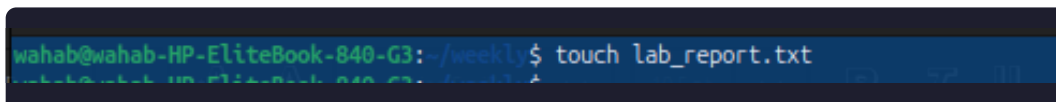
A terminal window showing the command `wahab@wahab-HP-EliteBook-840-G3:~/weekly$ touch lab_report.txt` being executed. The prompt is `wahab@wahab-HP-EliteBook-840-G3:~/weekly$`.

Figure 1 — Executing 'touch lab_report.txt' to create an empty file.

Observations:

- The command executes silently (no output), which is normal — touch returns success without printing anything.
- The working directory shown is the home folder of user *wahab*.

Step 2 — View Default Permissions (ls -l)

Running **ls -l** displays the long listing format which includes the permission string, link count, owner, group, size, date, and filename.

```
$ ls -l lab_report.txt
```

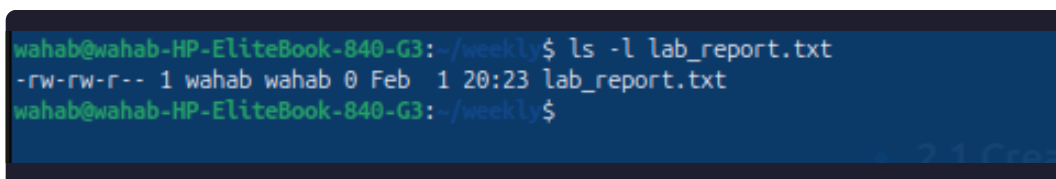
A terminal window showing the command `wahab@wahab-HP-EliteBook-840-G3:~/weekly$ ls -l lab_report.txt` being executed. The output is `-rw-rw-r-- 1 wahab wahab 0 Feb 1 20:23 lab_report.txt`. The prompt is `wahab@wahab-HP-EliteBook-840-G3:~/weekly$`.

Figure 2 — Default permission string '-rw-r--r--' shown by ls -l.

Segment	Bits	Meaning
File type	-	Regular file (not a directory or link)
Owner (wahab)	rw-	Read + Write; cannot execute
Group (wahab)	r--	Read only
Others	r--	Read only

Observations:

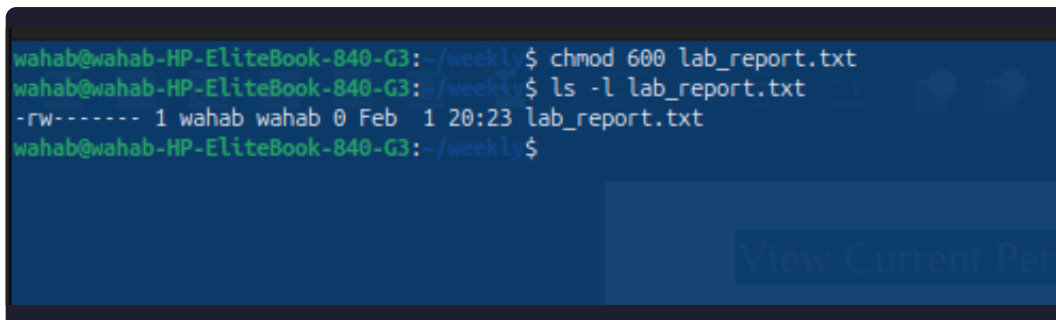
- Default permission for a newly created file is **644** (rw-r--r--).
- The owner has read and write access; group and others have read-only access.
- No execute bit is set — text files are not executable by default.

Step 3 — Modifying Permissions with chmod

Task A — Secure the File (Owner Only, chmod 600)

Setting permissions to **600** grants the owner read and write access while completely removing all access from group and others. This is the standard setting for private key files and sensitive documents.

```
$ chmod 600 lab_report.txt
```



```
wahab@wahab-HP-EliteBook-840-G3:~/weekly$ chmod 600 lab_report.txt
wahab@wahab-HP-EliteBook-840-G3:~/weekly$ ls -l lab_report.txt
-rw----- 1 wahab wahab 0 Feb  1 20:23 lab_report.txt
wahab@wahab-HP-EliteBook-840-G3:~/weekly$
```

Figure 3 — After 'chmod 600': permission string becomes '-rw-----'.

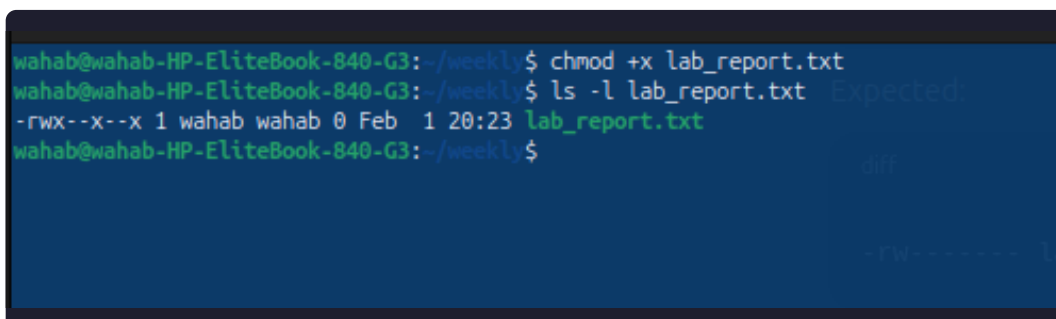
Observations:

- Permission string changed from **-rw-r--r--** to **-rw-----**.
- Octal 6 = binary 110 = Read(4) + Write(2). Group and Others get 0 (no access).
- This is the most secure setting for a single-user private file.

Task B — Make the File Executable (chmod +x)

Adding the execute bit with **chmod +x** enables the file to be run as a program or script. The terminal typically highlights executable files in green for visual identification.

```
$ chmod +x lab_report.txt
```



```
wahab@wahab-HP-EliteBook-840-G3:~/weekly$ chmod +x lab_report.txt
wahab@wahab-HP-EliteBook-840-G3:~/weekly$ ls -l lab_report.txt
-rwx--x--x 1 wahab wahab 0 Feb  1 20:23 lab_report.txt
wahab@wahab-HP-EliteBook-840-G3:~/weekly$
```

Figure 4 — After 'chmod +x': permission string becomes '-rwx--x--x'.

Observations:

- Execute bit (x) added to Owner, Group, and Others simultaneously.
- Permission string shows **-rwx--x--x** — owner can read/write/execute; others can only execute.
- The filename may appear highlighted (green) in the terminal when ls is run with colour support.

Task C — Full Access to Everyone (chmod 777)

Setting **777** grants full read, write, and execute permissions to the owner, group, and all other users on the system. This is used for testing purposes only and is considered a security risk in production.

```
$ chmod 777 lab_report.txt
```

```
wahab@wahab-HP-EliteBook-840-G3:~/weekly$ chmod +x lab_report.txt
wahab@wahab-HP-EliteBook-840-G3:~/weekly$ ls -l lab_report.txt
-rwx--x--x 1 wahab wahab 0 Feb  1 20:23 lab_report.txt
wahab@wahab-HP-EliteBook-840-G3:~/weekly$ chmod 777 lab_report.txt
wahab@wahab-HP-EliteBook-840-G3:~/weekly$ ls -l lab_report.txt
-rwxrwxrwx 1 wahab wahab 0 Feb  1 20:23 lab_report.txt
wahab@wahab-HP-EliteBook-840-G3:~/weekly$
```

Figure 5 — After 'chmod 777': permission string becomes '-rwxrwxrwx'. Both chmod 600 and chmod 777 steps shown sequentially.

Observations:

- Permission string is **-rwxrwxrwx**: all three permission sets fully open.
- Octal 7 = binary 111 = Read(4) + Write(2) + Execute(1).
- **Security Note:** chmod 777 is highly discouraged on shared systems. Any user can modify or delete the file.

Step 4 — Change File Ownership (chown)

The **chown** command changes who owns a file. Because changing ownership is a privileged operation, it must be prefixed with **sudo** to execute with root (administrator) privileges. A password prompt will appear.

```
$ sudo chown root lab_report.txt
$ ls -l lab_report.txt
```

```
wahab@wahab-HP-EliteBook-840-G3:~/weekly$ chmod +x lab_report.txt
wahab@wahab-HP-EliteBook-840-G3:~/weekly$ ls -l lab_report.txt
-rwx--x--x 1 wahab wahab 0 Feb  1 20:23 lab_report.txt
wahab@wahab-HP-EliteBook-840-G3:~/weekly$ chmod 777 lab_report.txt
wahab@wahab-HP-EliteBook-840-G3:~/weekly$ ls -l lab_report.txt
-rwxrwxrwx 1 wahab wahab 0 Feb  1 20:23 lab_report.txt
wahab@wahab-HP-EliteBook-840-G3:~/weekly$ sudo chown root lab_report.txt
[sudo] password for wahab:
wahab@wahab-HP-EliteBook-840-G3:~/weekly$ ls -l lab_report.txt
-rwxrwxrwx 1 root wahab 0 Feb  1 20:23 lab_report.txt
wahab@wahab-HP-EliteBook-840-G3:~/weekly$
```

Figure 6 — After 'sudo chown root': owner field changes from 'wahab' to 'root'.

Observations:

- The sudo password prompt appears as '[sudo] password for wahab:'.
- After authentication, the owner field in ls -l changes from **wahab** to **root**.
- The group field remains **wahab** because chown without a colon only changes the owner.
- Once owned by root, regular users cannot modify the file without sudo.

Task 2: Using Communication Tools

Linux provides several built-in tools for inter-user communication directly through the terminal. In this task we demonstrate TTY-based direct messaging and remote login via SSH.

Step 1 — Direct Terminal Messaging (TTY Redirect)

Two terminal windows are opened side-by-side. Each window is assigned a pseudo-terminal device (`pts/N`). The `tty` command identifies which device a terminal is attached to. A message is then redirected using the shell's output redirection operator (`>`) directly to the other terminal's device file.

```
$ tty
$ echo "Hello from Terminal A!" > /dev/pts/1
```

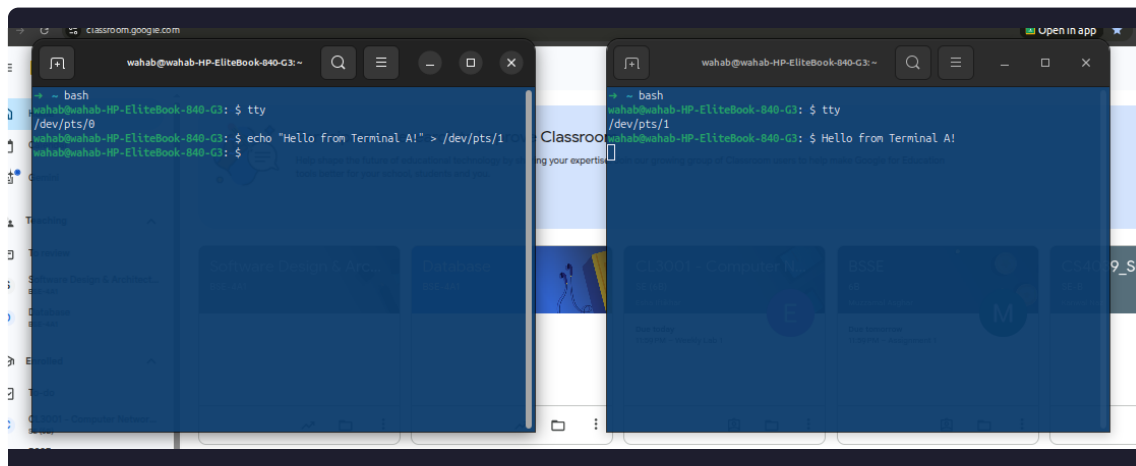


Figure 7 — Two terminals side-by-side. Terminal A (`pts/0`) sends a message to Terminal B (`pts/1`) using echo redirection. The message appears instantly in Terminal B.

Observations:

- Terminal A identifies itself as `/dev/pts/0` using the `tty` command.
- Terminal B identifies itself as `/dev/pts/1`.
- The message 'Hello from Terminal A!' is redirected to `/dev/pts/1` and appears immediately in Terminal B's display.
- This demonstrates that in Linux, devices (including terminals) are represented as files — a core Unix philosophy.

Step 2 — Remote Login via SSH (`ssh localhost`)

Secure Shell (SSH) is the standard protocol for encrypted remote login. To test it locally, we install the OpenSSH server and connect to our own machine using `ssh localhost`. On first connection, SSH presents a host key fingerprint for verification.

```
$ sudo apt update && sudo apt install openssh-server
$ ssh localhost
```

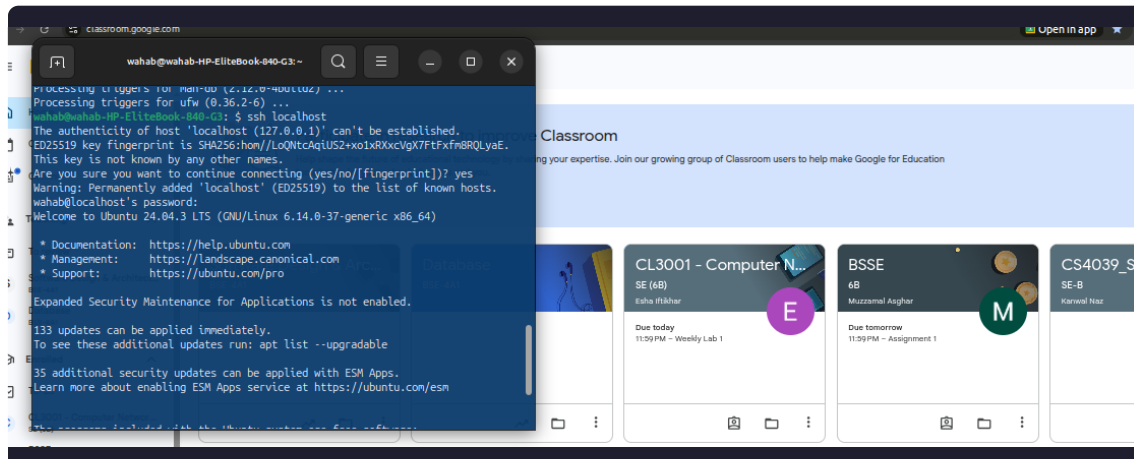


Figure 8 — SSH connecting to localhost. Host fingerprint verification prompt, password authentication, and successful Ubuntu 24.04 login banner.

Observations:

- SSH displays the SHA256 host key fingerprint for **127.0.0.1** on first connection.
- The user types 'yes' to add localhost to the list of known hosts (`~/.ssh/known_hosts`).
- After password authentication, the full Ubuntu 24.04 LTS MOTD (Message of the Day) is displayed, confirming a successful login session.
- 133 pending package updates and 35 security updates are visible in the banner — a real-world indicator that the system requires maintenance.

Summary of Commands Used

Command	Purpose	Key Outcome
<code>touch lab_report.txt</code>	Create empty file	New file with 644 default perms
<code>ls -l lab_report.txt</code>	List permissions	Displays <code>rw-</code> permission string
<code>chmod 600 lab_report.txt</code>	Owner-only access	String: <code>-rw-----</code>
<code>chmod +x lab_report.txt</code>	Add execute bit	String: <code>-rwx--x--x</code>
<code>chmod 777 lab_report.txt</code>	Full access (testing)	String: <code>-rwxrwxrwx</code>
<code>sudo chown root ...</code>	Change owner to root	Owner field: <code>wahab</code> → <code>root</code>
<code>tty</code>	Identify terminal device	Returns <code>/dev/pts/N</code>
<code>echo ... > /dev/pts/1</code>	Send message to terminal	Message appears in target terminal
<code>apt install openssh-server</code>	Install SSH server	<code>sshd</code> service enabled
<code>ssh localhost</code>	Remote login locally	Authenticated SSH session on self

Conclusion

This laboratory successfully demonstrated the core Linux permission model using **chmod** (numeric and symbolic modes) and **chown** for ownership management. The experiments confirmed that Linux enforces strict access control at the filesystem level, offering granular control over who can read, write, or execute a file.

The communication section illustrated two complementary approaches: lightweight terminal-to-terminal messaging via TTY device file redirection, and encrypted remote login via the SSH protocol. These foundational skills form the basis of Linux system administration and network security.